

MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

**Mini Project Report
of
Database Management System Lab (MCA 4141)**

TOY MANUFACTURING AND IMPORT DEPENDENCY ANALYSIS SYSTEM

SUBMITTED BY

STUDENT NAME	REG. NO.	SECTION	BATCH

**School of Computer Engineering
Manipal Institute of Technology, Manipal.
October 2026**

CERTIFICATE

This is to certify that the project titled “Toy Manufacturing and Import Dependency Analysis System” is a record of the bona fide work done by Student(s) (Reg. No. _____) submitted in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications of Manipal Institute of Technology, Manipal, Karnataka, during the academic year 2026–2027.

Name	and	Signature	of	Examiners:
------	-----	-----------	----	------------

1.				
----	--	--	--	--

2.				
----	--	--	--	--

TABLE OF CONTENTS

SECTION	PAGE
ABSTRACT	
CHAPTER 1: INTRODUCTION	
CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES	
CHAPTER 3: FUNCTIONAL REQUIREMENTS – MODULES AND SOFTWARE USED	
CHAPTER 4: ER DIAGRAM & RELATIONAL TABLES ALONG WITH SAMPLE DATA	
CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS	
CHAPTER 6: RESULTS & SNAPSHOTS	
CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK	
CHAPTER 8: REFERENCES	

Page numbers may be updated in Microsoft Word after final formatting and entry of student details.

ABSTRACT

The Toy Manufacturing and Import Dependency Analysis System is a database-oriented mini project designed to manage and analyze the lifecycle of toy manufacturing activities. The system maintains structured information about toys, raw materials, suppliers, sourcing options, manufacturing batches, inventory, sales channels, and sales transactions. The proposed database supports the operational flow from raw material sourcing to toy manufacturing, inventory maintenance, sales, and profit analysis.

The distinctive feature of the proposed system is its Import Dependency and Local Substitution Impact Analysis. The system identifies raw materials sourced through imported suppliers, determines whether locally available alternatives exist, and compares their costs. This allows the manufacturer to evaluate the possible impact of local substitution on manufacturing cost and potential profitability. The system does not assume that local sourcing is always cheaper; instead, it provides a data-driven comparison.

The project is intended for implementation using Oracle SQL and PL/SQL and demonstrates relational database design, normalization, constraints, joins, aggregate functions, nested queries, views, indexes, sequences, stored procedures, functions, and triggers. The scope is intentionally limited so that it remains a feasible DBMS mini project rather than a complete enterprise resource planning system.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Toy manufacturing involves interconnected activities such as raw material procurement, supplier selection, production, stock management, and sales. When these activities are maintained through separate records, it becomes difficult to obtain a consolidated view of production cost, sourcing information, inventory, and sales performance.

1.2 Interpretation of the Original Abstract

The original abstract requires a system that can provide information about toys, imported sources, raw materials, manufacturing stages, e-commerce sales, profit analysis, and reduction of imported goods. If interpreted as a complete business application, this would require supply-chain management, procurement, logistics, accounting, and external platform integration. Such a system would be too large for a DBMS mini project. Therefore, the proposed project refines the abstract into a focused relational database and decision-support system.

1.3 Need for the Proposed System

- Centralized storage of toy, material, supplier, production, inventory, and sales data.
- Traceability from raw material sourcing to finished toy sales.
- Identification of imported and local sourcing options.
- Manufacturing cost and profit analysis through SQL queries.
- A practical method to analyze local alternatives for imported materials.

1.4 Scope of the Project

The project covers raw material sourcing, toy material requirements, manufacturing batches, inventory, sales channels, sales, profit analysis, and import dependency analysis. It excludes real-time e-commerce APIs, online payments, customer management, logistics tracking, customs processing, demand forecasting, machine learning, and complete ERP functionality.

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Toy manufacturers require a structured mechanism for maintaining information about toys, raw materials, suppliers, manufacturing activities, inventory, and sales. Dependence on imported raw materials may influence production cost and sourcing decisions. When sourcing information is not integrated with toy requirements and manufacturing records, it becomes difficult to determine which toys depend on imported materials, whether local alternatives exist, and how a sourcing change may affect cost and profit.

2.2 Objectives

1. Maintain information about toys and their categories.
2. Maintain a master database of raw materials.
3. Define the materials and quantities required for each toy.
4. Maintain supplier information and supplier-material sourcing options.
5. Classify sourcing options as LOCAL or IMPORTED.
6. Record country of origin, unit cost, and availability of supplied materials.
7. Track manufacturing batches and actual material usage.
8. Maintain current finished toy inventory.
9. Record sales through multiple sales channels.
10. Calculate manufacturing cost, revenue, and profit.
11. Analyze import dependency and local substitution opportunities.
12. Compare imported and local sourcing costs.

2.3 Existing System

A simplified existing approach may rely on spreadsheets, manual registers, and disconnected files. Supplier data, production records, stock, and sales information may be maintained separately. This can lead to data redundancy, inconsistent updates, difficulty in tracing relationships, and manual calculation of cost and profitability.

2.4 Proposed System

The proposed system integrates the relevant manufacturing lifecycle information into a normalized Oracle database. Relational tables and foreign keys connect suppliers, raw materials, toys, manufacturing batches, inventory, sales channels, and sales transactions. SQL queries and views provide analytical reports, while selected PL/SQL procedures, functions, and triggers implement useful business logic.

2.5 Novelty / Unique Feature

The Import Dependency and Local Substitution Impact Analysis is the project's principal novelty. A raw material can have multiple supplier-material records. Some may be imported and others local. The system compares available sourcing options for the same material and estimates the resulting cost difference when multiplied by the quantity required for a toy. This makes the project a database-based decision-support system rather than only an inventory or manufacturing management system.

CHAPTER 3: FUNCTIONAL REQUIREMENTS – MODULES AND SOFTWARE USED

3.1 Modules of the System

MODULE	OBJECTIVE	TABLES	OUTPUT
Toy Management	Maintain toy master information.	TOY	Toy list and category report
Raw Material & BOM	Maintain materials and toy-wise requirements.	RAW_MATERIAL, TOY_MATERIAL	Material requirement report
Supplier & Sourcing	Maintain suppliers, source type, origin, cost and availability.	SUPPLIER, SUPPLIER_MATERIAL	Supplier and sourcing reports
Manufacturing Tracking	Record batches and actual material usage.	MANUFACTURING_BATCH, BATCH_MATERIAL_USAGE	Batch history and cost
Inventory Management	Maintain finished toy stock.	INVENTORY	Current and low-stock report
Sales & Channel Tracking	Record sales through channels.	SALES_CHANNEL, SALE	Channel-wise sales
Profit Analysis	Analyze revenue, cost and profit.	SALE, MANUFACTURING_BATCH, BATCH_MATERIAL_USAGE	Toy-wise profitability
Import Dependency Analysis	Compare imported sourcing and local alternatives.	SUPPLIER_MATERIAL, TOY_MATERIAL	Dependency and substitution reports

3.2 Functional Requirements

- Add, update, and retrieve toy records.
- Add and maintain raw material records.
- Map multiple materials to a toy through a bill-of-materials relationship.
- Allow multiple suppliers for the same material.
- Store source type, country of origin, availability, and unit cost.
- Record manufacturing batches and material consumption.
- Maintain current inventory.
- Record sales and associated sales channels.
- Generate manufacturing, inventory, sales, profit, and sourcing analysis reports.

3.3 Non-Functional Requirements

- Data integrity through primary keys, foreign keys, and CHECK constraints.
- Normalized design to reduce redundancy.
- Implementation using Oracle SQL and PL/SQL.
- Simple and maintainable structure suitable for academic demonstration.
- No dependence on external APIs or advanced infrastructure.

3.4 Hardware and Software Requirements

CATEGORY	REQUIREMENT
Operating System	Windows or another OS supported by Oracle

	development tools
Database	Oracle Database / Oracle XE where appropriate
Development Tool	Oracle SQL Developer or equivalent
Languages	SQL and PL/SQL
Documentation	Microsoft Word or compatible software
Hardware	Standard laptop/desktop capable of running Oracle development tools

3.5 Overall System Workflow

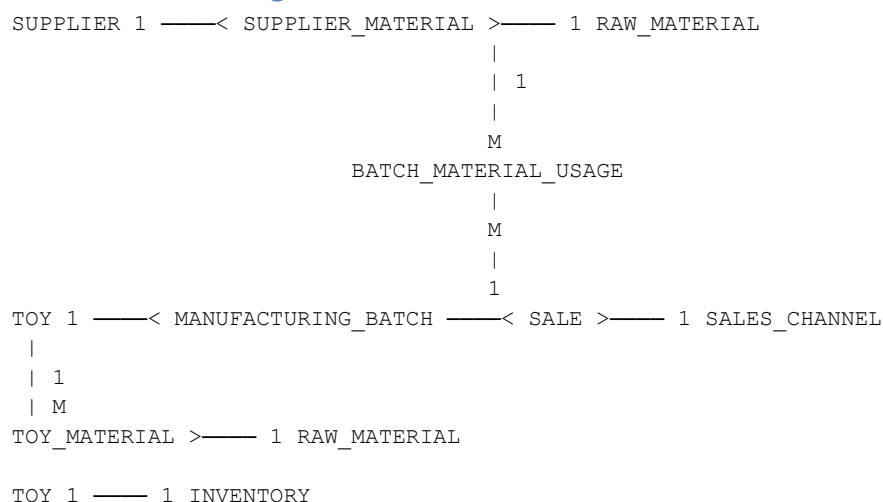
Supplier / Source → Raw Material → Toy Material Requirement → Manufacturing Batch → Inventory → Sales Channel and Sale → Profit Analysis. The novelty feature operates on sourcing and material requirement data to identify imported dependence and compare available local alternatives.

CHAPTER 4: ER DIAGRAM & RELATIONAL TABLES ALONG WITH SAMPLE DATA

4.1 Final Entities

13. TOY
14. RAW_MATERIAL
15. SUPPLIER
16. SUPPLIER_MATERIAL
17. TOY_MATERIAL
18. MANUFACTURING_BATCH
19. BATCH_MATERIAL_USAGE
20. INVENTORY
21. SALES_CHANNEL
22. SALE

4.2 Textual ER Diagram



4.3 Relationship Explanation

RELATIONSHIP	CARDINALITY	DESCRIPTION
TOY → TOY MATERIAL	1:M	A toy requires multiple materials.
RAW_MATERIAL → TOY MATERIAL	1:M	A material can be used by multiple toys.
SUPPLIER → SUPPLIER MATERIAL	1:M	A supplier can provide multiple materials.
RAW_MATERIAL → SUPPLIER MATERIAL	1:M	A material can have multiple suppliers.
TOY → MANUFACTURING BATCH	1:M	A toy can be produced in multiple batches.
MANUFACTURING_BATCH → BATCH MATERIAL USAGE	1:M	A batch can have multiple material usage records.
SUPPLIER_MATERIAL → BATCH MATERIAL USAGE	1:M	A sourcing option can be used in multiple batches.
TOY → INVENTORY	1:1 logical current stock	One current inventory record is maintained per toy.

MANUFACTURING_BATCH → SALE	1:M	A batch can be associated with multiple sales.
SALES_CHANNEL → SALE	1:M	A channel can contain multiple sale records.

4.4 Table Structure

TABLE	PURPOSE	PRIMARY / FOREIGN KEYS
TOY	Stores toy master information.	PK toy_id
RAW_MATERIAL	Stores material master information.	PK material_id
SUPPLIER	Stores supplier details.	PK supplier_id
SUPPLIER_MATERIAL	Maps suppliers and materials and stores sourcing details.	PK supplier_material_id; FK supplier_id; FK material_id
TOY_MATERIAL	Maps toys and required materials.	PK toy_material_id; FK toy_id; FK material_id
MANUFACTURING_BATCH	Stores production batch information.	PK batch_id; FK toy_id
BATCH_MATERIAL_USAGE	Stores actual material usage and historical cost.	PK batch_usage_id; FK batch_id; FK supplier_material_id
INVENTORY	Stores current finished toy stock.	PK inventory_id; FK toy_id UNIQUE
SALES_CHANNEL	Stores sales platform information.	PK channel_id
SALE	Stores sales transactions.	PK sale_id; FK batch_id; FK channel_id

4.5 Normalization

The database is designed approximately in Third Normal Form (3NF). Atomic values are stored separately, many-to-many relationships are resolved through TOY_MATERIAL and SUPPLIER_MATERIAL, and non-key attributes depend on the appropriate entity key. Supplier details are not repeated in material records, and historical batch material cost is preserved separately from current supplier cost.

4.6 Sample Data

TOY ID	TOY NAME	CATEGORY	SELLING PRICE
1	Remote Control Car	Electronic	1499.00
2	Wooden Building Blocks	Educational	899.00
3	Musical Robot	Electronic	1999.00

MATERIAL	SOURCE TYPE	ORIGIN	UNIT COST
ABS Plastic	IMPORTED	China	120.00
ABS Plastic	LOCAL	India	105.00
DC Motor	IMPORTED	China	300.00
DC Motor	LOCAL	India	325.00
Wooden Block	LOCAL	India	35.00

CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

5.1 DDL Commands

```
CREATE TABLE TOY (  
    toy_id NUMBER PRIMARY KEY,  
    toy_name VARCHAR2(100) NOT NULL,  
    category VARCHAR2(50),  
    description VARCHAR2(500),  
    recommended_age VARCHAR2(30),  
    standard_selling_price NUMBER(10,2)  
        CHECK (standard_selling_price > 0),  
    status VARCHAR2(20) DEFAULT 'ACTIVE'  
        CHECK (status IN ('ACTIVE','INACTIVE'))  
);  
  
CREATE TABLE RAW_MATERIAL (  
    material_id NUMBER PRIMARY KEY,  
    material_name VARCHAR2(100) NOT NULL UNIQUE,  
    material_category VARCHAR2(50),  
    unit VARCHAR2(20) NOT NULL,  
    description VARCHAR2(300)  
);  
  
CREATE TABLE SUPPLIER (  
    supplier_id NUMBER PRIMARY KEY,  
    supplier_name VARCHAR2(100) NOT NULL,  
    country VARCHAR2(50) NOT NULL,  
    contact_person VARCHAR2(100),  
    phone VARCHAR2(20),  
    email VARCHAR2(100)  
);  
  
CREATE TABLE SUPPLIER_MATERIAL (  
    supplier_material_id NUMBER PRIMARY KEY,  
    supplier_id NUMBER NOT NULL,  
    material_id NUMBER NOT NULL,  
    source_type VARCHAR2(15) NOT NULL  
        CHECK (source_type IN ('LOCAL','IMPORTED')),  
    country_of_origin VARCHAR2(50) NOT NULL,  
    unit_cost NUMBER(10,2) NOT NULL CHECK (unit_cost > 0),  
    availability_status VARCHAR2(20) DEFAULT 'AVAILABLE'  
        CHECK (availability_status IN ('AVAILABLE','UNAVAILABLE')),  
    FOREIGN KEY (supplier_id) REFERENCES SUPPLIER(supplier_id),  
    FOREIGN KEY (material_id) REFERENCES RAW_MATERIAL(material_id)  
);  
  
CREATE TABLE TOY_MATERIAL (  
    toy_material_id NUMBER PRIMARY KEY,  
    toy_id NUMBER NOT NULL,  
    material_id NUMBER NOT NULL,  
    quantity_required NUMBER(10,2) NOT NULL  
        CHECK (quantity_required > 0),  
    unit VARCHAR2(20) NOT NULL,  
    UNIQUE (toy_id, material_id),  
    FOREIGN KEY (toy_id) REFERENCES TOY(toy_id),  
    FOREIGN KEY (material_id) REFERENCES RAW_MATERIAL(material_id)  
);
```

```

CREATE TABLE MANUFACTURING_BATCH (
    batch_id NUMBER PRIMARY KEY,
    toy_id NUMBER NOT NULL,
    manufacturing_date DATE DEFAULT SYSDATE,
    quantity_produced NUMBER NOT NULL CHECK (quantity_produced > 0),
    labor_cost NUMBER(12,2) DEFAULT 0 CHECK (labor_cost >= 0),
    overhead_cost NUMBER(12,2) DEFAULT 0 CHECK (overhead_cost >= 0),
    batch_status VARCHAR2(20) DEFAULT 'COMPLETED'
    CHECK (batch_status IN ('PLANNED','IN_PROGRESS','COMPLETED')),
    FOREIGN KEY (toy_id) REFERENCES TOY(toy_id)
);

CREATE TABLE BATCH_MATERIAL_USAGE (
    batch_usage_id NUMBER PRIMARY KEY,
    batch_id NUMBER NOT NULL,
    supplier_material_id NUMBER NOT NULL,
    quantity_used NUMBER(10,2) NOT NULL CHECK (quantity_used > 0),
    actual_unit_cost NUMBER(10,2) NOT NULL CHECK (actual_unit_cost > 0),
    FOREIGN KEY (batch_id) REFERENCES MANUFACTURING_BATCH(batch_id),
    FOREIGN KEY (supplier_material_id)
        REFERENCES SUPPLIER_MATERIAL(supplier_material_id)
);

CREATE TABLE INVENTORY (
    inventory_id NUMBER PRIMARY KEY,
    toy_id NUMBER NOT NULL UNIQUE,
    quantity_available NUMBER DEFAULT 0 CHECK (quantity_available >= 0),
    reorder_level NUMBER DEFAULT 10 CHECK (reorder_level >= 0),
    last_updated DATE DEFAULT SYSDATE,
    FOREIGN KEY (toy_id) REFERENCES TOY(toy_id)
);

CREATE TABLE SALES_CHANNEL (
    channel_id NUMBER PRIMARY KEY,
    channel_name VARCHAR2(100) NOT NULL UNIQUE,
    channel_type VARCHAR2(30),
    commission_rate NUMBER(5,2) DEFAULT 0
    CHECK (commission_rate BETWEEN 0 AND 100)
);

CREATE TABLE SALE (
    sale_id NUMBER PRIMARY KEY,
    batch_id NUMBER NOT NULL,
    channel_id NUMBER NOT NULL,
    sale_date DATE DEFAULT SYSDATE,
    quantity_sold NUMBER NOT NULL CHECK (quantity_sold > 0),
    unit_selling_price NUMBER(10,2) NOT NULL CHECK (unit_selling_price > 0),
    discount_amount NUMBER(10,2) DEFAULT 0 CHECK (discount_amount >= 0),
    platform_fee NUMBER(10,2) DEFAULT 0 CHECK (platform_fee >= 0),
    FOREIGN KEY (batch_id) REFERENCES MANUFACTURING_BATCH(batch_id),
    FOREIGN KEY (channel_id) REFERENCES SALES_CHANNEL(channel_id)
);

```

5.2 Sequences and Indexes

```

CREATE SEQUENCE seq_toy START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_material START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_supplier START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_batch START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_inventory START WITH 1 INCREMENT BY 1;

```

```

CREATE SEQUENCE seq_sale START WITH 1 INCREMENT BY 1;

CREATE INDEX idx_toy_material_toy ON TOY_MATERIAL(toy_id);
CREATE INDEX idx_supplier_material_material
    ON SUPPLIER_MATERIAL(material_id);
CREATE INDEX idx_batch_toy ON MANUFACTURING_BATCH(toy_id);
CREATE INDEX idx_sale_date ON SALE(sale_date);

```

5.3 View – Batch Cost Analysis

```

CREATE OR REPLACE VIEW VW_BATCH_COST AS
SELECT b.batch_id, t.toy_name, b.quantity_produced,
       NVL(SUM(u.quantity_used * u.actual_unit_cost),0)
           AS material_cost,
       b.labor_cost,
       b.overhead_cost,
       NVL(SUM(u.quantity_used * u.actual_unit_cost),0)
           + b.labor_cost + b.overhead_cost AS total_batch_cost
FROM MANUFACTURING_BATCH b
JOIN TOY t ON t.toy_id = b.toy_id
LEFT JOIN BATCH_MATERIAL_USAGE u
    ON u.batch_id = b.batch_id
GROUP BY b.batch_id, t.toy_name, b.quantity_produced,
       b.labor_cost, b.overhead_cost;

```

5.4 Procedure – Add Manufacturing Batch

```

CREATE OR REPLACE PROCEDURE ADD_MANUFACTURING_BATCH (
    p_batch_id IN NUMBER,
    p_toy_id IN NUMBER,
    p_quantity IN NUMBER,
    p_labor_cost IN NUMBER,
    p_overhead_cost IN NUMBER
) AS
BEGIN
    INSERT INTO MANUFACTURING_BATCH
    VALUES (p_batch_id, p_toy_id, SYSDATE, p_quantity,
            p_labor_cost, p_overhead_cost, 'COMPLETED');

    MERGE INTO INVENTORY i
    USING (SELECT p_toy_id toy_id FROM dual) s
    ON (i.toy_id = s.toy_id)
    WHEN MATCHED THEN UPDATE SET
        quantity_available = i.quantity_available + p_quantity,
        last_updated = SYSDATE
    WHEN NOT MATCHED THEN INSERT
        (inventory_id, toy_id, quantity_available,
         reorder_level, last_updated)
    VALUES
        (seq_inventory.NEXTVAL, p_toy_id, p_quantity,
         10, SYSDATE);
END;
/

```

5.5 Function – Calculate Batch Cost

```

CREATE OR REPLACE FUNCTION CALCULATE_BATCH_COST (
    p_batch_id IN NUMBER
) RETURN NUMBER AS
    v_material NUMBER := 0;
    v_labor NUMBER := 0;
    v_overhead NUMBER := 0;

```

```

BEGIN
    SELECT NVL(SUM(quantity_used * actual_unit_cost),0)
    INTO v_material
    FROM BATCH_MATERIAL_USAGE
    WHERE batch_id = p_batch_id;

    SELECT labor_cost, overhead_cost
    INTO v_labor, v_overhead
    FROM MANUFACTURING_BATCH
    WHERE batch_id = p_batch_id;

    RETURN v_material + v_labor + v_overhead;
END;
/

```

5.6 Trigger – Prevent Invalid Sales and Update Inventory

```

CREATE OR REPLACE TRIGGER TRG_VALIDATE_SALE_INVENTORY
BEFORE INSERT ON SALE
FOR EACH ROW
DECLARE
    v_toy_id NUMBER;
    v_available NUMBER;
BEGIN
    SELECT toy_id INTO v_toy_id
    FROM MANUFACTURING_BATCH
    WHERE batch_id = :NEW.batch_id;

    SELECT quantity_available INTO v_available
    FROM INVENTORY
    WHERE toy_id = v_toy_id
    FOR UPDATE;

    IF :NEW.quantity_sold > v_available THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Insufficient inventory for this sale.');
```

END IF;

```

END;
/

CREATE OR REPLACE TRIGGER TRG_UPDATE_INVENTORY_AFTER_SALE
AFTER INSERT ON SALE
FOR EACH ROW
DECLARE
    v_toy_id NUMBER;
BEGIN
    SELECT toy_id INTO v_toy_id
    FROM MANUFACTURING_BATCH
    WHERE batch_id = :NEW.batch_id;

    UPDATE INVENTORY
    SET quantity_available =
        quantity_available - :NEW.quantity_sold,
        last_updated = SYSDATE
    WHERE toy_id = v_toy_id;
END;
/

```

CHAPTER 6: RESULTS & SNAPSHOTS

This chapter lists the principal SQL reports to be demonstrated during project evaluation. After implementing the database in Oracle SQL Developer, screenshots of the query output, views, procedures, functions, and trigger behavior can be inserted into this chapter.

6.1 Important SQL Reports

REPORT	DBMS CONCEPT
Raw materials required for a selected toy	JOIN
Supplier-wise material supply	Multiple JOIN
Imported versus local sourcing options	CASE, GROUP BY
Toys dependent on imported materials	JOIN and filtering
Materials with both local and imported suppliers	Conditional aggregation
Manufacturing history of a toy	JOIN and ORDER BY
Manufacturing cost of each batch	SUM and calculated columns
Toy-wise manufacturing cost	Aggregate functions
Current inventory	JOIN
Low-stock toys	Conditional filtering
Sales by e-commerce platform	JOIN, SUM, GROUP BY
Toy-wise revenue	Calculated aggregate
Toy-wise profit	View / aggregation
Most profitable toy	Nested query / ordering
Local substitution impact	Conditional aggregation and cost comparison

6.2 Example Query – Raw Materials Required for a Toy

```
SELECT t.toy_name, rm.material_name,
       tm.quantity_required, tm.unit
FROM TOY t
JOIN TOY_MATERIAL tm ON t.toy_id = tm.toy_id
JOIN RAW_MATERIAL rm ON tm.material_id = rm.material_id
WHERE t.toy_name = 'Remote Control Car';
```

6.3 Example Query – Imported versus Local Options

```
SELECT rm.material_name,
       SUM(CASE WHEN sm.source_type = 'IMPORTED'
                THEN 1 ELSE 0 END) AS imported_options,
       SUM(CASE WHEN sm.source_type = 'LOCAL'
                THEN 1 ELSE 0 END) AS local_options
FROM RAW_MATERIAL rm
JOIN SUPPLIER_MATERIAL sm
  ON sm.material_id = rm.material_id
GROUP BY rm.material_name;
```

6.4 Example Query – Current Inventory

```
SELECT t.toy_name, i.quantity_available,
       i.reorder_level,
       CASE WHEN i.quantity_available <= i.reorder_level
            THEN 'REORDER REQUIRED'
            ELSE 'SUFFICIENT STOCK'
       END AS inventory_status
FROM TOY t
JOIN INVENTORY i ON i.toy_id = t.toy_id;
```

6.5 Example Query – Sales by Channel

```
SELECT sc.channel_name,
       SUM(s.quantity_sold) AS total_units_sold,
       SUM(s.quantity_sold * s.unit_selling_price
           - s.discount_amount) AS gross_revenue
FROM SALE s
JOIN SALES_CHANNEL sc
  ON sc.channel_id = s.channel_id
GROUP BY sc.channel_name
ORDER BY gross_revenue DESC;
```

6.6 Novelty Report – Local Substitution Impact

```
SELECT t.toy_name,
       rm.material_name,
       tm.quantity_required,
       MIN(CASE WHEN sm.source_type = 'IMPORTED'
                THEN sm.unit_cost END) AS imported_cost,
       MIN(CASE WHEN sm.source_type = 'LOCAL'
                THEN sm.unit_cost END) AS local_cost,
       (
         MIN(CASE WHEN sm.source_type = 'LOCAL'
                  THEN sm.unit_cost END)
         -
         MIN(CASE WHEN sm.source_type = 'IMPORTED'
                  THEN sm.unit_cost END)
       ) * tm.quantity_required
       AS estimated_cost_difference
FROM TOY t
JOIN TOY_MATERIAL tm ON tm.toy_id = t.toy_id
JOIN RAW_MATERIAL rm ON rm.material_id = tm.material_id
JOIN SUPPLIER_MATERIAL sm ON sm.material_id = rm.material_id
WHERE sm.availability_status = 'AVAILABLE'
GROUP BY t.toy_name, rm.material_name,
         tm.quantity_required
HAVING MIN(CASE WHEN sm.source_type = 'IMPORTED'
                  THEN sm.unit_cost END) IS NOT NULL
       AND MIN(CASE WHEN sm.source_type = 'LOCAL'
                  THEN sm.unit_cost END) IS NOT NULL;
```

6.7 Expected Interpretation

MATERIAL	IMPORTED COST	LOCAL COST	INTERPRETATION
ABS Plastic	₹120	₹105	Local option reduces cost by ₹15 per unit.
DC Motor	₹300	₹325	Local option exists but increases cost by ₹25 per unit.

This report demonstrates the practical value of the novelty feature: the database provides evidence for sourcing decisions instead of assuming that local sourcing is always economically superior.

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusion

The Toy Manufacturing and Import Dependency Analysis System demonstrates how a relational database can represent the important stages of toy manufacturing in a structured and manageable manner. The design integrates toys, raw materials, suppliers, sourcing options, manufacturing batches, inventory, sales channels, and sales transactions through normalized relationships.

The project's distinctive contribution is its Import Dependency and Local Substitution Impact Analysis. The feature extends the system beyond ordinary inventory management by identifying imported sourcing dependence, checking for locally available alternatives, and comparing their potential cost impact. This directly connects the project to the original abstract while remaining feasible for a DBMS laboratory mini project.

7.2 Limitations

- Manual data entry is assumed.
- No real-time e-commerce API integration.
- No automated supplier price updates.
- No demand forecasting or machine learning.
- No logistics, shipment, or customs tracking.
- No complete accounting or ERP functionality.

7.3 Future Work

- Supplier performance analysis.
- Multi-warehouse inventory.
- Purchase order and procurement management.
- Demand forecasting and production planning.
- Dashboard-based analytical visualization.
- Real-time sales platform integration.
- Automated sourcing recommendations based on cost and availability.

7.4 Feasibility Study

Technical Feasibility: The project is technically feasible using Oracle SQL, PL/SQL, and Oracle SQL Developer. All major functions can be implemented using standard relational database concepts without advanced infrastructure.

Operational Feasibility: The proposed database can be useful to small toy manufacturers or entrepreneurs who require structured information about production, materials, suppliers, inventory, sales, and sourcing alternatives.

Economic Feasibility: The project has a low academic implementation cost because it requires no paid APIs, cloud services, or external infrastructure. Sample data can be created locally.

Project Scope Feasibility: The use of ten main tables and a limited set of procedures, functions, views, and triggers provides sufficient depth for evaluation while remaining manageable within a mini-project timeline.

CHAPTER 8: REFERENCES

1. Oracle. Oracle Database SQL Language Reference. Oracle Database Documentation.
2. Oracle. PL/SQL Language Reference. Oracle Database Documentation.
3. Elmasri, R., and Navathe, S. B. Fundamentals of Database Systems.
4. Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts.
5. Coronel, C., and Morris, S. Database Systems: Design, Implementation, and Management.
6. Relevant Government of India and industry publications concerning the growth and development of the Indian toy manufacturing sector, where used for background discussion.

APPENDIX: VIVA PREPARATION

QUESTION	SUGGESTED ANSWER
What is the main purpose of the project?	To manage toy manufacturing data from sourcing through production, inventory, sales, profit, and import dependency analysis.
What is the novelty?	The system compares imported sourcing options with available local alternatives and estimates their cost impact.
Why is SUPPLIER_MATERIAL needed?	It resolves the many-to-many relationship between suppliers and materials and stores source-specific attributes.
Why is BATCH_MATERIAL_USAGE needed?	It records actual material usage and historical cost for a manufacturing batch.
Why store actual_unit_cost?	Current supplier prices can change; actual_unit_cost preserves the historical production cost.
Why is the database normalized?	Normalization reduces redundancy and update, insertion, and deletion anomalies.
Why is this not an ERP?	The scope intentionally focuses on database operations and excludes large enterprise modules.